

Guia do Arguente

SHOPISEL

Aplicação de Lista de Compras com Preços e Alertas



Martim Monteiro N°51619
Alexandre Tavares N°51271
Pedro Gomes N°51423

Orientadores: Doutor Fernando Miguel Gamboa de Carvalho, João Pereira

1 Junho de 2026

1 Identificação do projeto

Título do projeto: SHOPISEL – Aplicação de Lista de Compras com Preços e Alertas

Orientadores: Doutor Fernando Miguel Gamboa de Carvalho, João Pereira

2 Repositório e acesso ao código

URL da organização GitHub: <https://github.com/shopisel>

O projeto encontra-se dividido em vários repositórios dentro da organização GitHub indicada. Cada repositório corresponde a uma componente funcional da solução, permitindo uma separação clara entre frontend, serviços backend, scrapers e infraestrutura.

3 Descrição da organização do dossier

O projeto encontra-se organizado por componentes funcionais, separando aplicações cliente, serviços backend, scrapers, infraestrutura e relatório. Esta organização facilita a navegação no código e a avaliação independente de cada parte da solução.

Pasta	Conteúdo principal
frontend/	Aplicação web principal do ShopISEL, desenvolvida em React com Vite, responsável pela experiência de utilização no browser.
frontend-mobile/	Aplicação móvel em Expo React Native, com integração com Firebase para notificações push e funcionalidades orientadas ao uso em smartphone.
account-service/	Microserviço autenticado responsável pela conta do utilizador, sincronização inicial com o token, gestão de favoritos e registo de tokens FCM para push notifications.
list-service/	Microserviço autenticado de gestão de listas de compras, incluindo criação, consulta, edição, remoção e atualização dos itens de cada lista.
products-service/	Microserviço de produtos e categorias, com pesquisa por nome, filtragem por categoria ou identificadores e sugestão de produtos relacionados.
prices-service/	Microserviço responsável pelo armazenamento e consulta de preços de produtos por loja, permitindo obter, criar, atualizar e remover registos de preço.
stores-service/	Microserviço de lojas, usado para consultar e gerir estabelecimentos por identificador, marca ou nome, incluindo dados de localização.
matchqueue-service/	Serviço que processa resultados de scraping após a recolha, fazendo o emparelhamento de produtos, atualização de preços e enfileiramento de eventos associados.
fcm-notification-worker/	Worker dedicado ao envio/processamento de notificações push através do Firebase Cloud Messaging.

scraper-continente/	Scraper em .NET com Playwright que percorre categorias do Continente e recolhe produtos, disponibilidade, imagens e informação de preço.
scraper-pingodoce/	Scraper em .NET com Playwright para produtos do Pingo Doce, com exportação para JSON, persistência em MongoDB e capacidade de disparar o <code>matchqueue-service</code> .
all-stores-scraper/	Ferramenta de recolha de lojas físicas de retalhistas, com exportação em JSON, CSV e SQL para posterior carga na base de dados.
scraper-orchestrator/	Repositório de orquestração que executa os scrapers de diferentes insígnias em paralelo via GitHub Actions e aciona um worker final no fim do processo.
shopisel_custom_label/	Repositório auxiliar de personalização visual/ <i>branding</i> , atualmente usado para definir configurações de marca e ativos gráficos (por exemplo, variante visual Continente), não correspondendo a um microserviço backend.
keycloak/	Configuração de autenticação centralizada com Keycloak e respetivo deploy em Fly.io.
deploy/	Configurações de infraestrutura e publicação, incluindo gateway/proxy, ficheiros de deploy para Fly.io e definições para Render.
report/	Relatório principal e artefactos em $\text{\LaTeX}$ associados à documentação académica do projeto.

4 Ambiente de demonstração em Fly.io

Para além da execução local, os serviços principais da solução encontram-se publicados em Fly.io. Assim, o arguente pode validar a aplicação através do ambiente remoto, sem necessidade de executar todos os microserviços localmente.

URL do gateway em Fly.io: <https://shopisel-gateway-fly.fly.dev/>

Recomenda-se que a validação comece pelo ambiente publicado, utilizando a conta de demonstração indicada neste documento. A execução local pode ser utilizada posteriormente para análise técnica individual de cada componente.

5 Tecnologias principais

- Frontend web em React com Vite;
- Frontend mobile em Expo React Native;
- Backend em .NET, organizado em microserviços;
- Autenticação centralizada com Keycloak;
- Notificações push com Firebase Cloud Messaging;
- Deploy e encaminhamento via Fly.io;
- Recolha automática de dados com scrapers dedicados por insígnia.

6 Informação para instalação e execução local

6.1 Frontend web

- Entrar na pasta `frontend/`;
- Instalar dependências com `npm install`;
- Executar em desenvolvimento com `npm run dev`;
- Configurar o ficheiro de ambiente com base em `frontend/.env.example`.

6.2 Frontend mobile

- Entrar na pasta `frontend-mobile/`;
- Instalar dependências com `npm install`;
- Iniciar com `npx expo start -go`;
- Usar Expo Go ou emulador/dispositivo compatível para testes.

6.3 Serviços backend

Apesar de os serviços estarem disponíveis em Fly.io para demonstração, cada microserviço backend também pode ser executado individualmente em ambiente local. O procedimento base é o seguinte:

- entrar na pasta do serviço pretendido;
- executar `dotnet restore src`;
- executar `dotnet test src`, quando existirem testes;
- executar `dotnet run -project src/<NomeDoProjeto>`.

Os serviços autenticados requerem, pelo menos, configurações de `ConnectionStrings` e de integração com Keycloak, nomeadamente valores para `Authority`, `Audience` e outras chaves equivalentes definidas nos respetivos READMEs e `appsettings`.

```
ConnectionString: Host=ep-sparkling-tooth-abqgborn.eu-west-2.aws.neon.tech;Port=5432;Database=neondb;Username=neondb_owner;Password=npg_gJG0P9QjufFY;SSLMode=Require;TrustServerCertificate=true
```

A connection string de acesso à base de dados deve ser colocada no ficheiro `appsettings.Development.json` de cada serviço, na secção `ConnectionStrings`. Por exemplo, no caso do `list-service`, deve ser usada a chave `ConnectionStrings:ListService`.

6.4 Infraestrutura e autenticação

- O gateway e os upstreams encontram-se documentados nos ficheiros de configuração do Fly.io, nomeadamente em `deploy/fly.toml`;
- O Keycloak encontra-se configurado na pasta `keycloak/`;
- Para o Keycloak, devem ser definidos os segredos de administração e de acesso à base de dados indicados em `keycloak/README.md`;
- O sistema depende de PostgreSQL e, para algumas funcionalidades, de Firebase Cloud Messaging.

7 Credenciais de acesso

Para a demonstração e validação do sistema, o arguente pode utilizar a seguinte conta de teste:

Conta de demonstração: teste@gmail.com

Palavra-passe: 123

Caso seja necessário aceder à conta de administração do Keycloak, as respetivas credenciais deverão ser solicitadas posteriormente.

8 Guia de validação sugerido ao arguente

Para facilitar a análise do trabalho realizado, sugere-se o seguinte percurso de validação, preferencialmente através do ambiente publicado em Fly.io:

1. autenticar no sistema com a conta de demonstração;
2. criar ou editar uma lista de compras;
3. consultar produtos, categorias e preços;
4. validar a integração entre frontend, backend e autenticação;
5. observar a organização dos microserviços e da componente de scraping;
6. confirmar a existência da configuração de deploy e da documentação técnica.

9 Observações complementares

Este guia tem como finalidade apoiar a análise do projeto ShopISEL, fornecendo ao arguente os elementos essenciais para a validação funcional da solução, para a consulta da organização dos repositórios e para a compreensão da estrutura técnica do sistema.